Intermediate (timeouts and retry-with-backoff) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `retry_backoff_practice.py`: a network call with no `timeout=` can hang forever, and one that retries *instantly* on failure just hammers a struggling server. This exercise builds the retry-with-backoff loop — a timeout on every call, and a wait that doubles after each failure (`2 ** attempt`) — that becomes `http_client.py`'s core almost unchanged. **If not:** the Build Task's retry logic would be written from scratch under deadline pressure, with no smaller, already-working version to copy from — exactly the mistake this document's whole structure is built to avoid.

Solution

```
# retry_backoff_practice.py
import logging
import time
import requests

logger = logging.getLogger(__name__)

def get_with_retry(url: str, max_attempts: int = 5) -> requests.Response:
    for attempt in range(max_attempts):
        try:
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout:
            if attempt == max_attempts - 1:
                raise
            wait_seconds = 2 ** attempt
            logger.warning(
                "attempt %s timed out, waiting %ss", attempt, wait_seconds
            )
            time.sleep(wait_seconds)
    raise RuntimeError("unreachable")  # loop always returns or raises above

if __name__ == "__main__":
    try:
        requests.get("http://10.255.255.1", timeout=(1, 2))
    except requests.exceptions.Timeout:
```

```
        print("timed out, as expected")

    response = get_with_retry("https://api.github.com")
    print(response.status_code)
```

Expected output:

```
timed out, as expected
200
```